

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also studies Backpropagation Through Time (BPTT), vanishing and exploding gradients, sequence-length effects, and video understanding using CNN-extracted features followed by recurrent networks. A synthetic encoder–decoder sequence-to-sequence task is also implemented.

2. Learning Outcomes

After completing this experiment, the following were demonstrated:

- representation of sequential data using (N, T, F) format;
- implementation of SimpleRNN, LSTM and GRU for sequence classification;
- interpretation of BPTT and gradient-related limitations;
- comparison using accuracy, macro precision, recall and F1-score;
- analysis of training/validation curves and confusion matrices;
- construction of a CNN–LSTM/CNN–GRU video pipeline;
- understanding of encoder–decoder sequence-to-sequence learning; and
- comparison of predictive performance, model complexity and training cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones (HAR). The raw inertial signals were used rather than the precomputed 561-feature representation. Each activity window contains 128 temporal measurements and nine channels: three body-acceleration, three gyroscope and three total-acceleration signals. Therefore the input is represented as

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The six classes are WALKING, WALKING UPSTAIRS, WALKING DOWNSTAIRS, SITTING, STANDING and LAYING.

The executed notebook combined the available train and test windows and then created a stratified 70%/15%/15% train/validation/test split. The resulting tensors were:

Split	Number of sequences	Shape
Training	7209	(7209, 128, 9)
Validation	1545	(1545, 128, 9)
Testing	1545	(1545, 128, 9)

Normalization was performed using statistics fitted only on the training set.

4. Github Link

<https://github.com/PranavVarshin/DeepLearning/tree/main/Lab-6>

5. Temporal Data Visualization

Three representative activity classes were visualized using body acceleration X, body gyroscope X and total acceleration X against the time step.

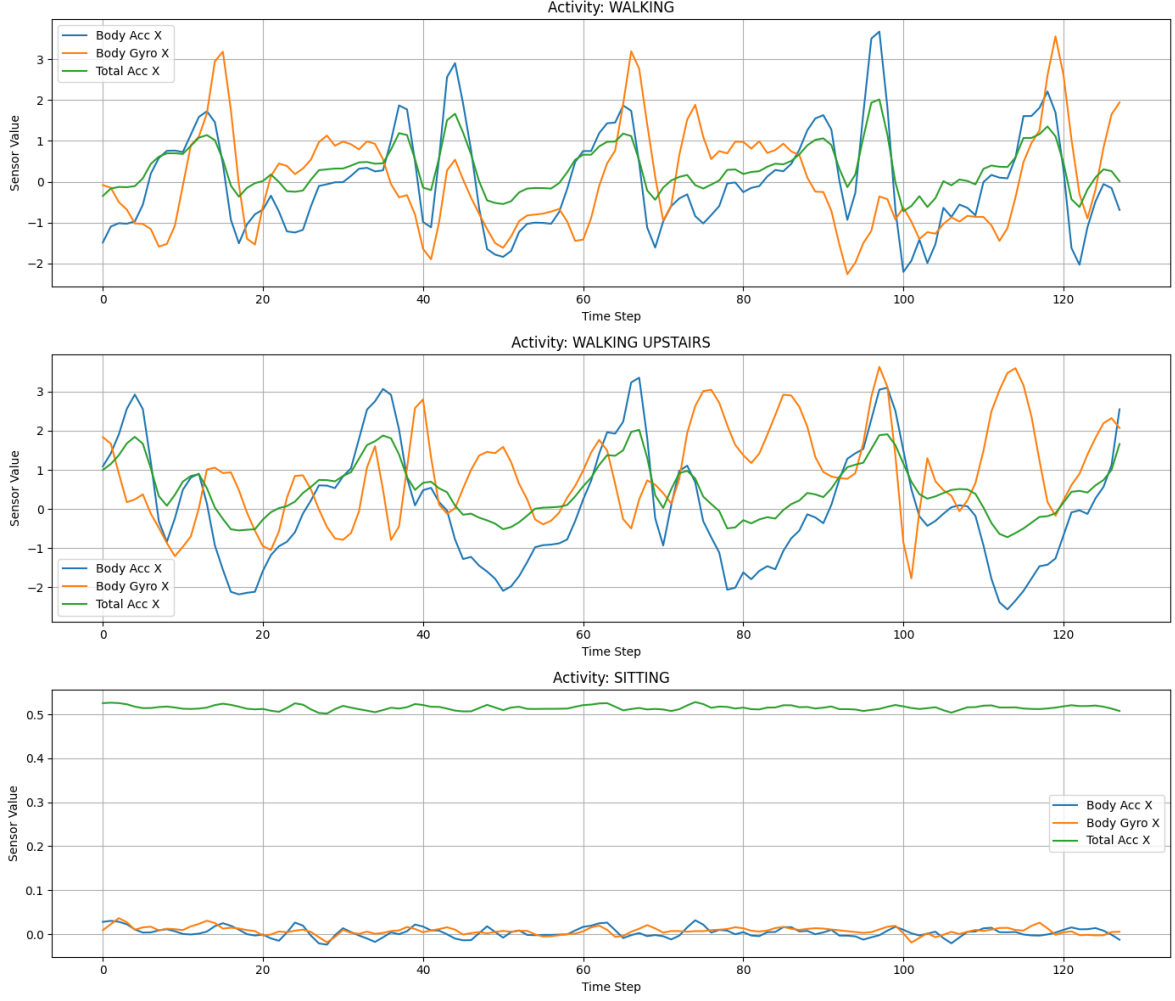


Figure 1: Representative temporal sensor signals for three activity classes.

Inference: The sensor measurements vary over time and the temporal pattern differs between activities. The ordering of observations is important because the same sensor values in a different temporal order can represent different motion dynamics. This temporal structure motivates the use of recurrent networks.

6. Vanilla RNN Architecture

A Vanilla RNN updates its hidden state according to

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h),$$

and the output is obtained from

$$y_t = g(W_y h_t + b_y).$$

The implemented classifier was

$$128 \times 9 \rightarrow \text{SimpleRNN}(32) \rightarrow \text{Dropout}(0.2) \rightarrow \text{Dense}(16, \text{ReLU}) \rightarrow \text{Dense}(6, \text{Softmax}).$$

Adam with learning rate 10^{-3} , batch size 32 and 30 epochs was used.

Backpropagation Through Time

When the recurrent network is unrolled, the loss gradient is propagated through multiple time steps. Repeated multiplication of derivatives can make gradients very small or very large, leading respectively to vanishing or exploding gradients. This makes learning long-term dependencies difficult.

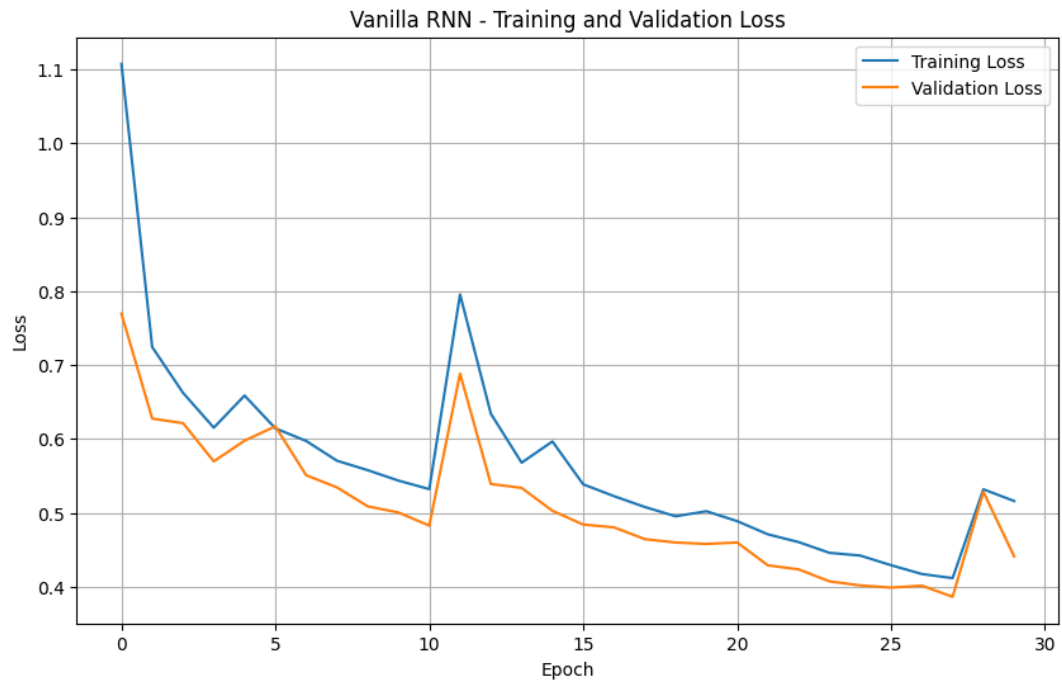


Figure 2: Vanilla RNN training and validation loss.

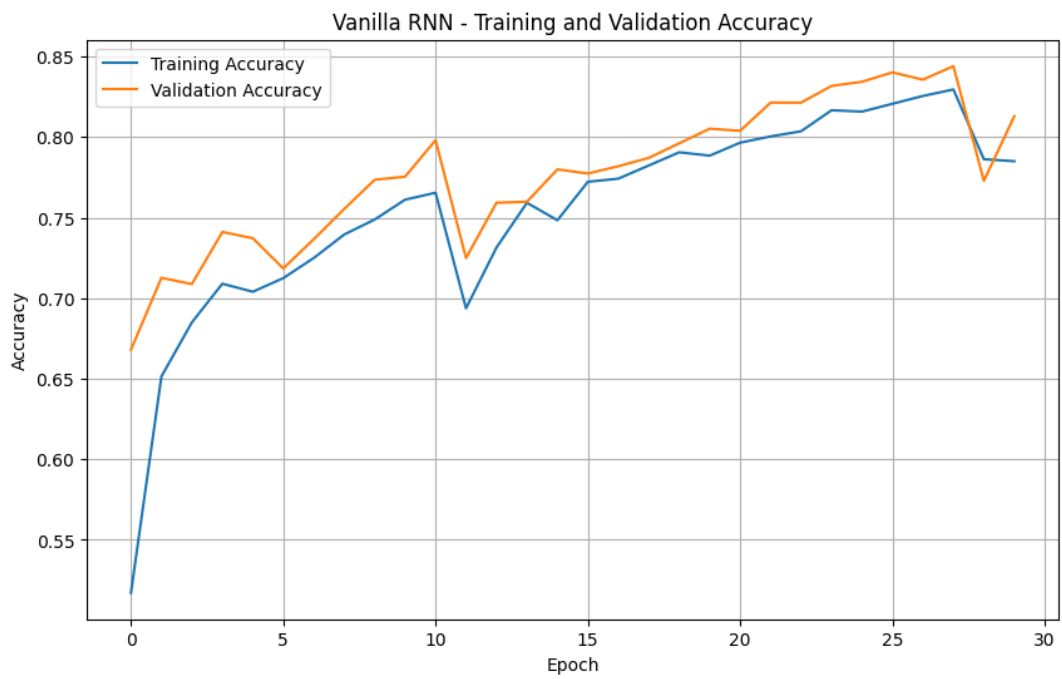


Figure 3: Vanilla RNN training and validation accuracy.

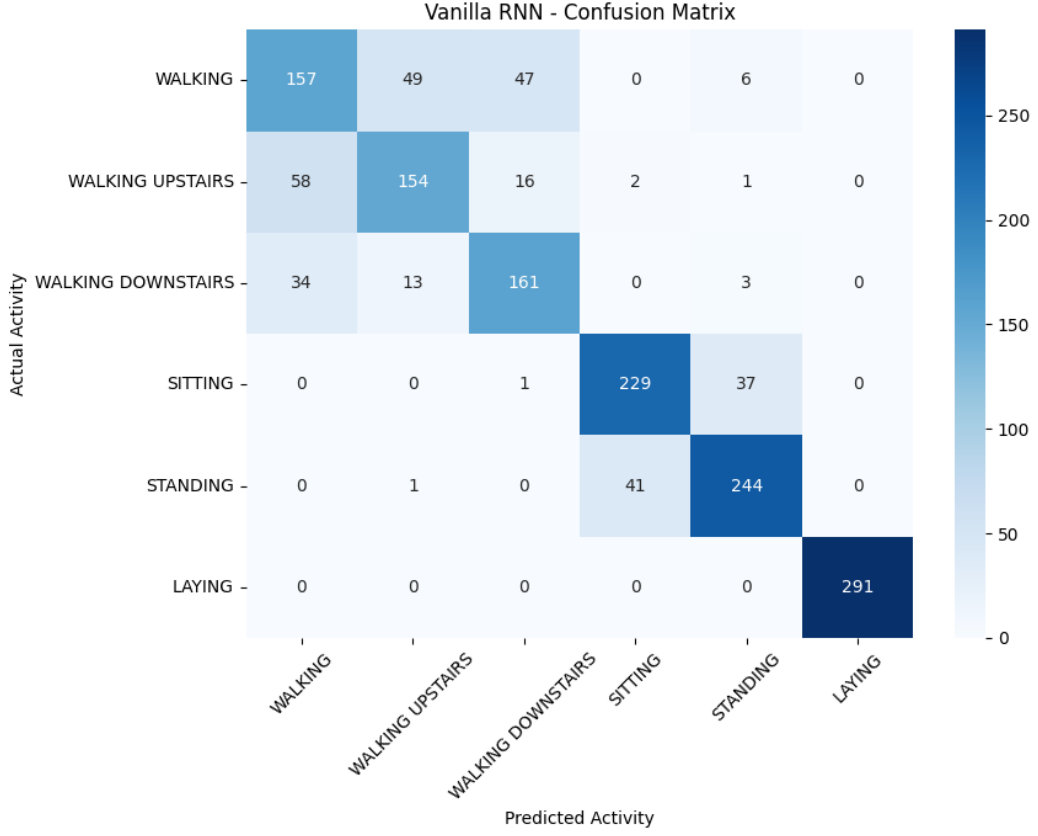


Figure 4: Vanilla RNN confusion matrix.

Inference: The RNN improves substantially during training, but its final test performance is lower than the gated recurrent models. The class-wise results show that LAYING is recognized very strongly, while dynamic activities such as WALKING and WALKING UPSTAIRS have more errors.

7. LSTM Architecture

LSTM introduces a memory cell and gates. The forget, input and output gates are

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell and hidden states are

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t, \quad h_t = o_t \tanh(C_t).$$

The same classifier structure was used with an LSTM layer of 32 units.

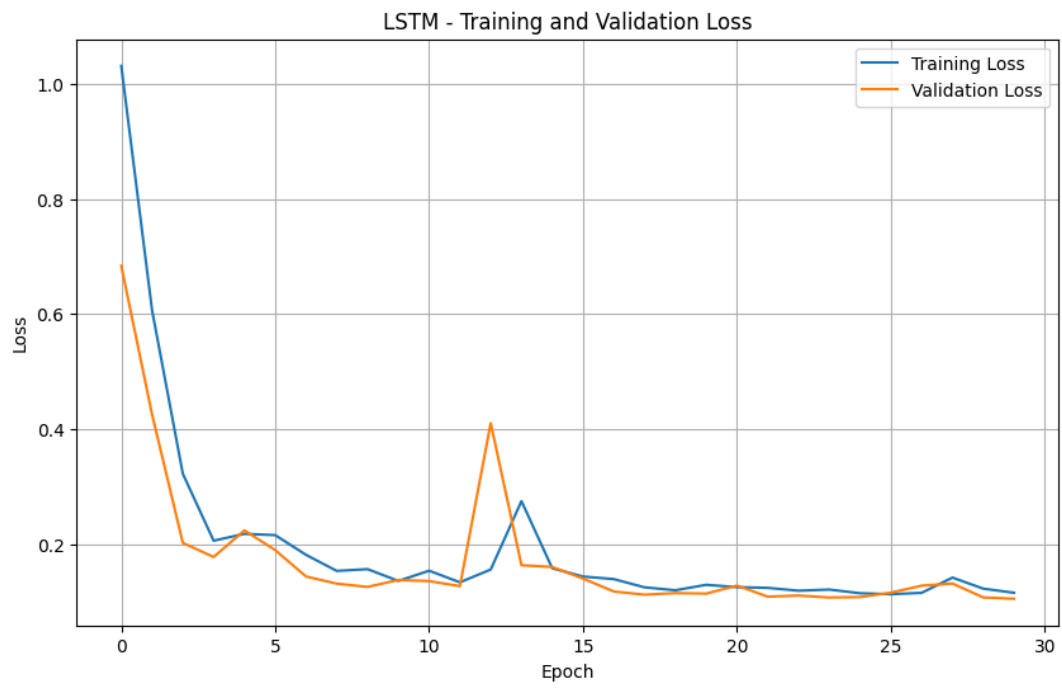


Figure 5: LSTM training and validation loss.

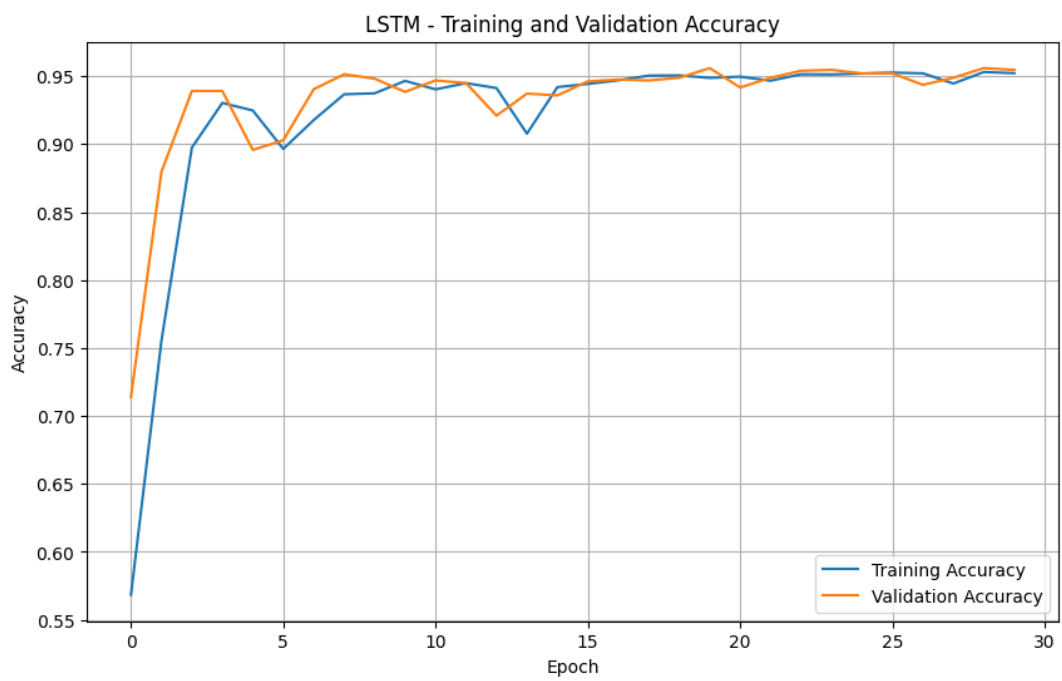


Figure 6: LSTM training and validation accuracy.

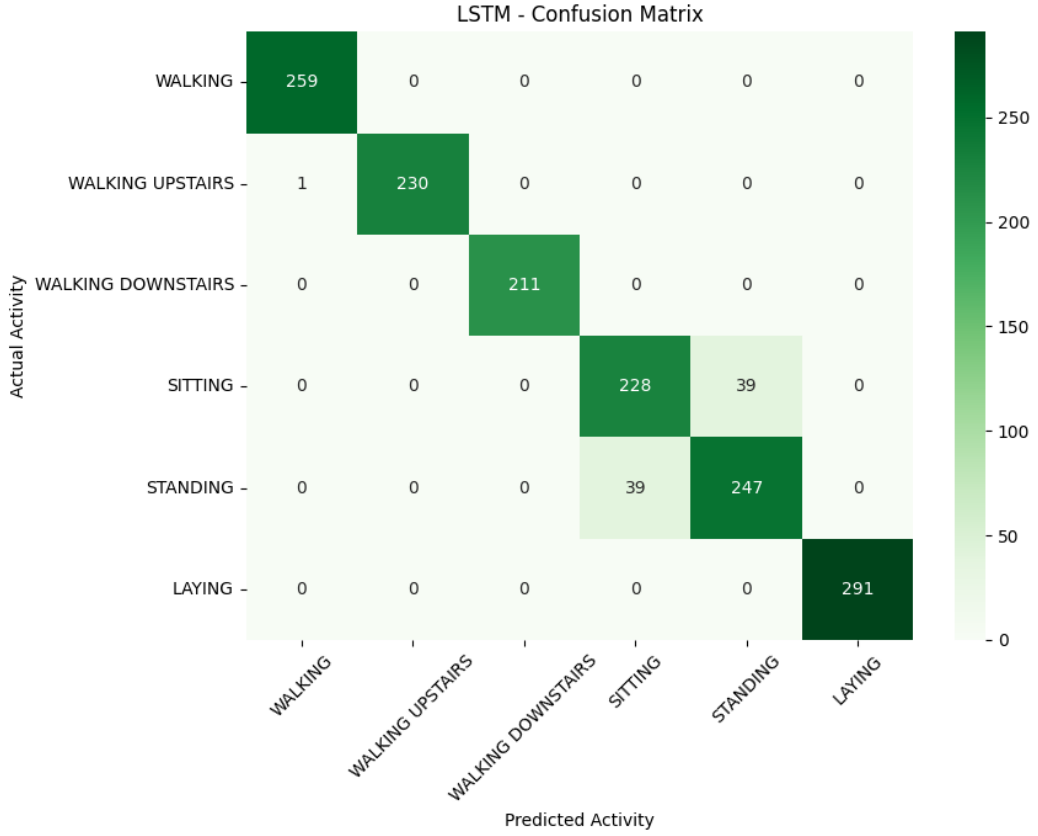


Figure 7: LSTM confusion matrix.

Test result: Test loss was 0.1211 and test accuracy was 94.89%. The macro precision, recall and F1-score were approximately 95.23%, 95.22% and 95.22%, respectively.

Inference: The LSTM reaches high validation accuracy early and maintains strong generalization. The cell state and gates provide a mechanism for selectively retaining or discarding information over the sequence.

8. GRU Architecture

GRU uses an update gate and reset gate:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z), \quad r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate state and final hidden state are

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h),$$

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$

The implemented GRU used 32 units with the same dropout, dense classifier, optimizer, batch size and epoch count as the RNN and LSTM.

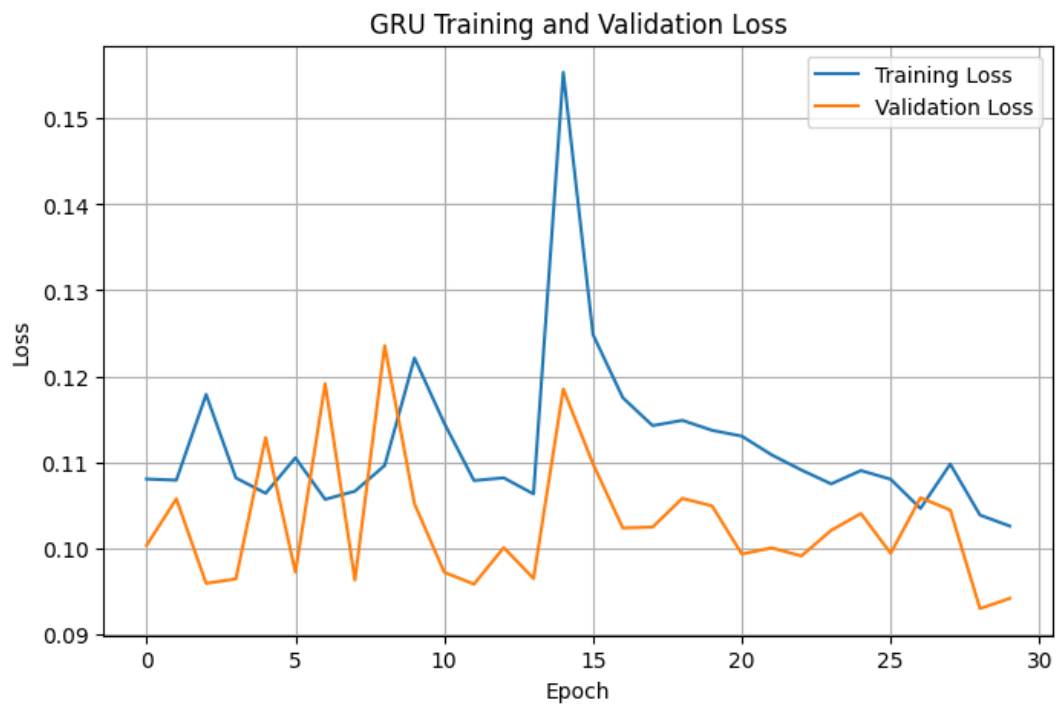


Figure 8: GRU training and validation loss.

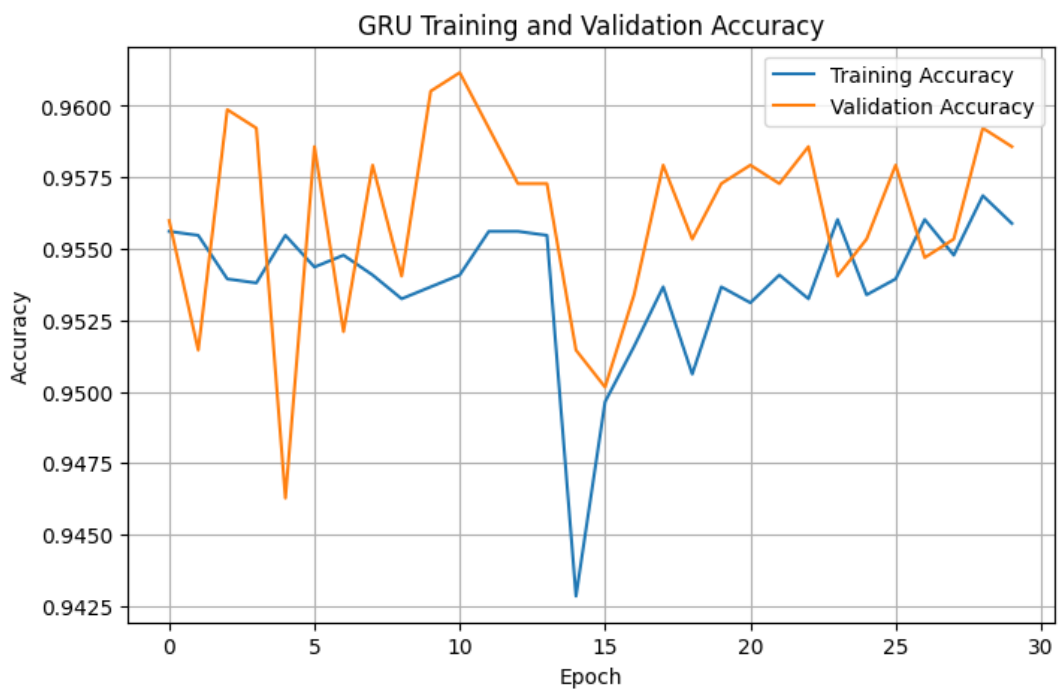


Figure 9: GRU training and validation accuracy.

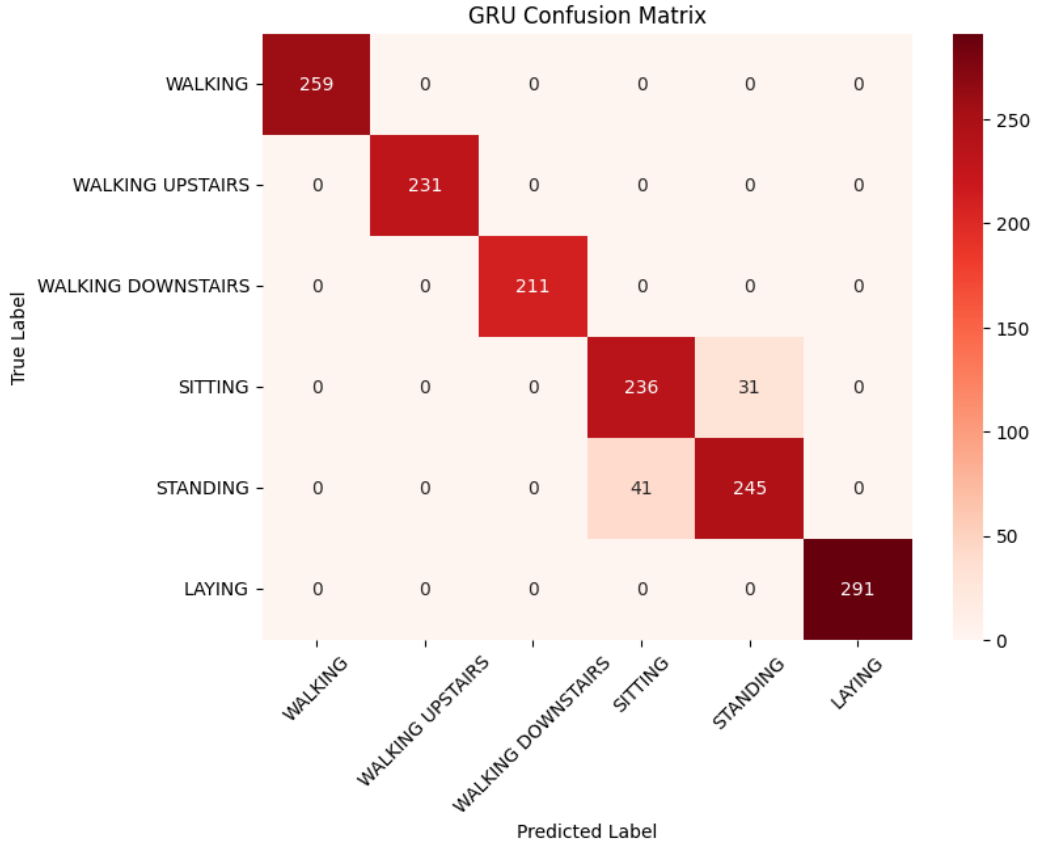


Figure 10: GRU confusion matrix.

Inference: The GRU maintains stable high validation accuracy throughout training. It uses fewer trainable parameters than the LSTM while achieving slightly higher test performance in this experiment.

9. RNN, LSTM and GRU Comparison

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Parameters	Time (s)
RNN	80.00%	78.94%	79.11%	78.99%	1,974	78.61
LSTM	94.89%	95.23%	95.22%	95.22%	6,006	78.86
GRU	95.34%	95.66%	95.68%	95.66%	4,758	73.95

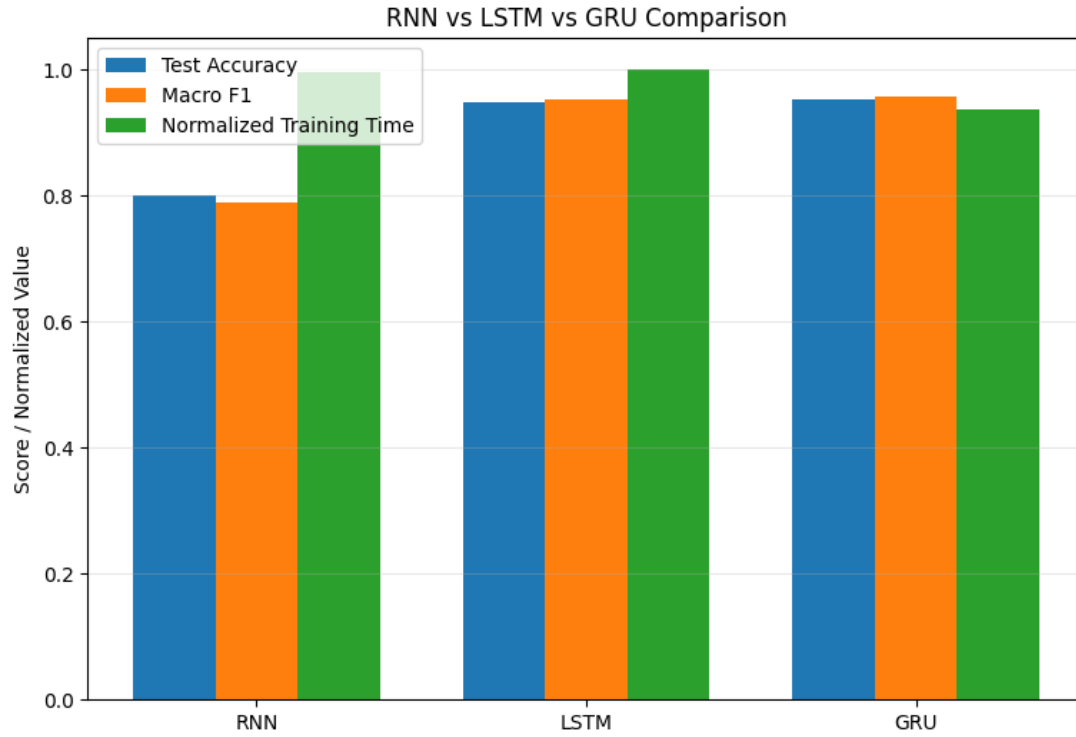


Figure 11: Comparison of recurrent-model test performance.

The RNN has the smallest parameter count but also the lowest observed performance. LSTM and GRU use gated memory mechanisms and substantially improve the classification metrics. GRU uses fewer parameters than LSTM and required less training time in this execution.

10. Effect of Sequence Length

The recurrent models were evaluated using $T \in \{32, 64, 128\}$ by consistently truncating each input sequence.

Sequence Length	Model	Test Accuracy	Macro F1
32	RNN	86.47%	86.17%
32	LSTM	94.05%	94.30%
32	GRU	93.98%	94.24%
64	RNN	72.82%	70.48%
64	LSTM	94.82%	95.11%
64	GRU	94.76%	95.08%
128	RNN	69.19%	68.36%
128	LSTM	94.50%	94.78%
128	GRU	95.02%	95.34%

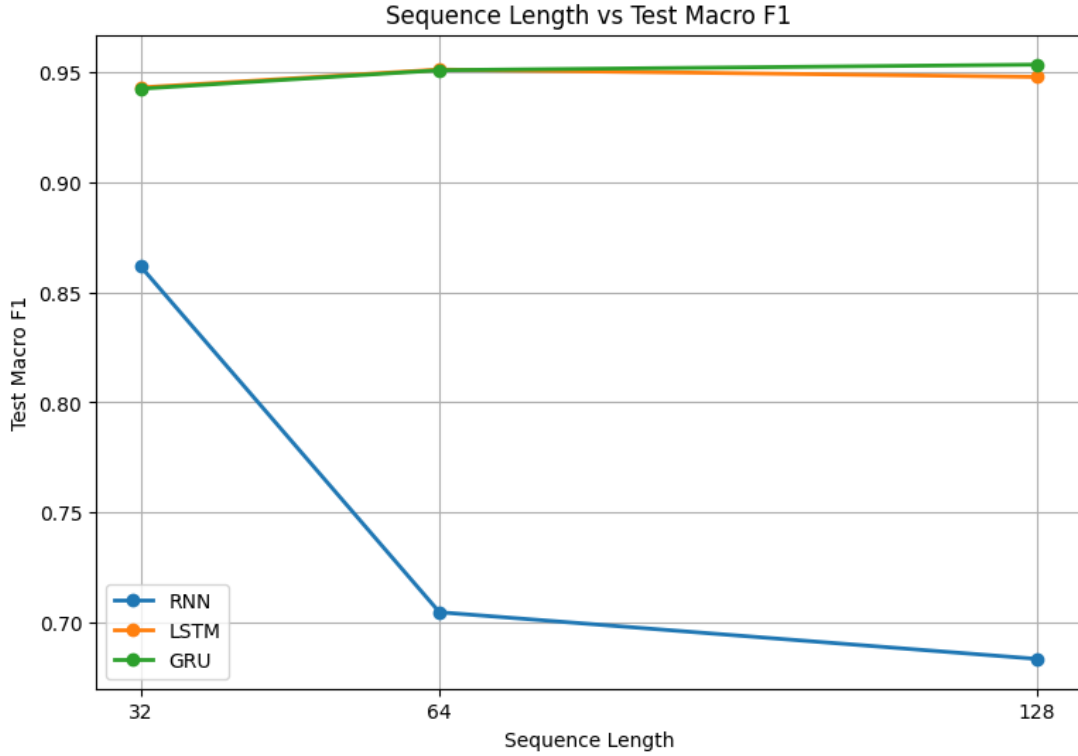


Figure 12: Effect of sequence length on test Macro F1-score.

Inference: The effect of sequence length is model-dependent. RNN performance decreased as the sequence became longer in this execution. LSTM achieved its highest Macro F1 at $T = 64$, while GRU achieved its highest Macro F1 at $T = 128$. Thus, additional temporal context did not affect all recurrent architectures in the same way.

11. Video Understanding Using CNN and RNN

A video was treated as a sequence of frames. A pretrained MobileNetV2 was used as a frozen CNN feature extractor, followed by an LSTM or GRU classifier. Three UCF101 classes were used: ApplyEyeMakeup, Archery and BabyCrawling.

Ten frames were sampled uniformly from each video and resized to $224 \times 224 \times 3$. MobileNetV2 with ImageNet weights, the classification head removed and global average pooling enabled, produced a 1280-dimensional feature vector per frame. Therefore each video was represented as

$$X \in \mathbb{R}^{10 \times 1280}.$$

A total of 121 videos were processed, producing the split:

Split	Videos	Shape
Training	84	(84, 10, 1280)
Validation	18	(18, 10, 1280)
Testing	19	(19, 10, 1280)

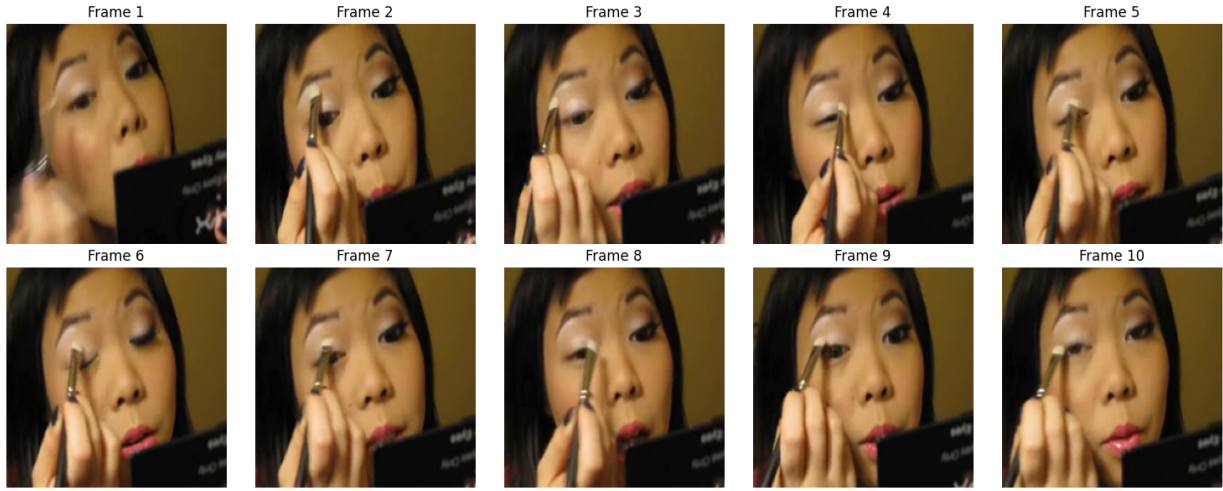


Figure 13: Ten uniformly sampled frames from a representative UCF101 video.

Spatial-temporal interpretation: MobileNetV2 extracts spatial information such as visual patterns and object/action-related features from each frame. The recurrent network receives the ordered frame features and models their temporal relationships.

CNN-LSTM

The architecture was

$$10 \times 1280 \rightarrow \text{LSTM}(32) \rightarrow \text{Dropout}(0.2) \rightarrow \text{Dense}(16) \rightarrow \text{Softmax}(3).$$

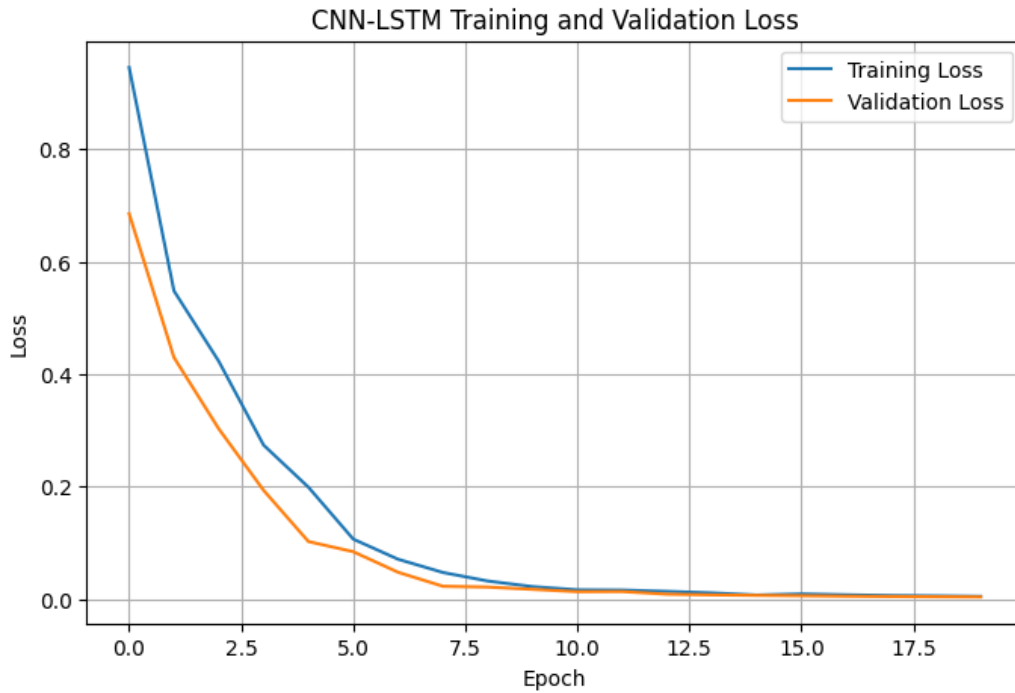


Figure 14: CNN-LSTM training and validation loss.

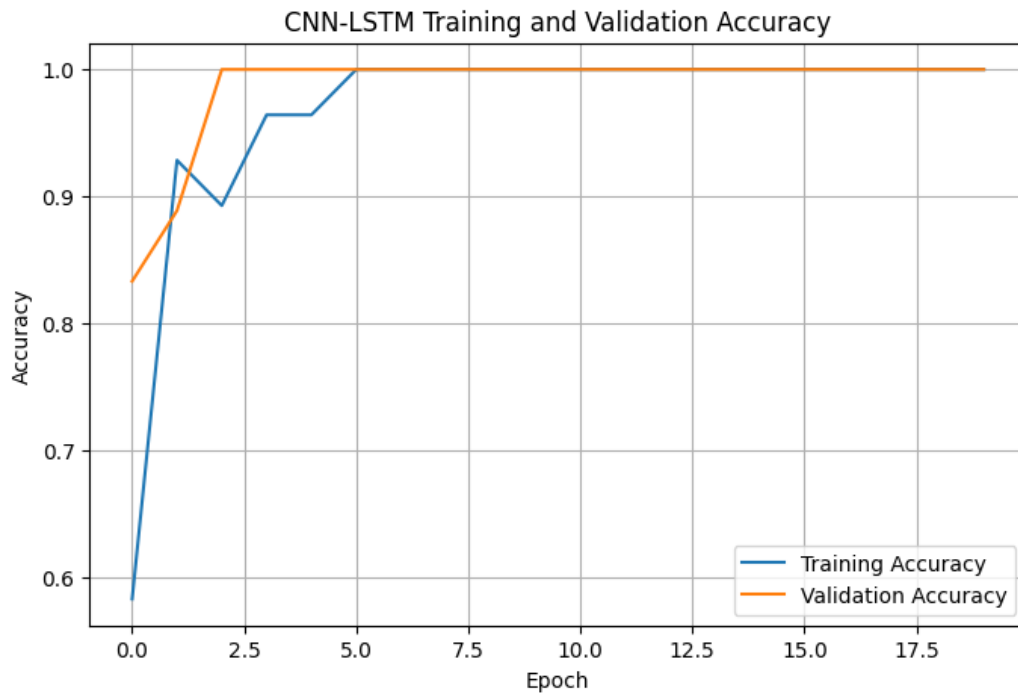


Figure 15: CNN-LSTM training and validation accuracy.

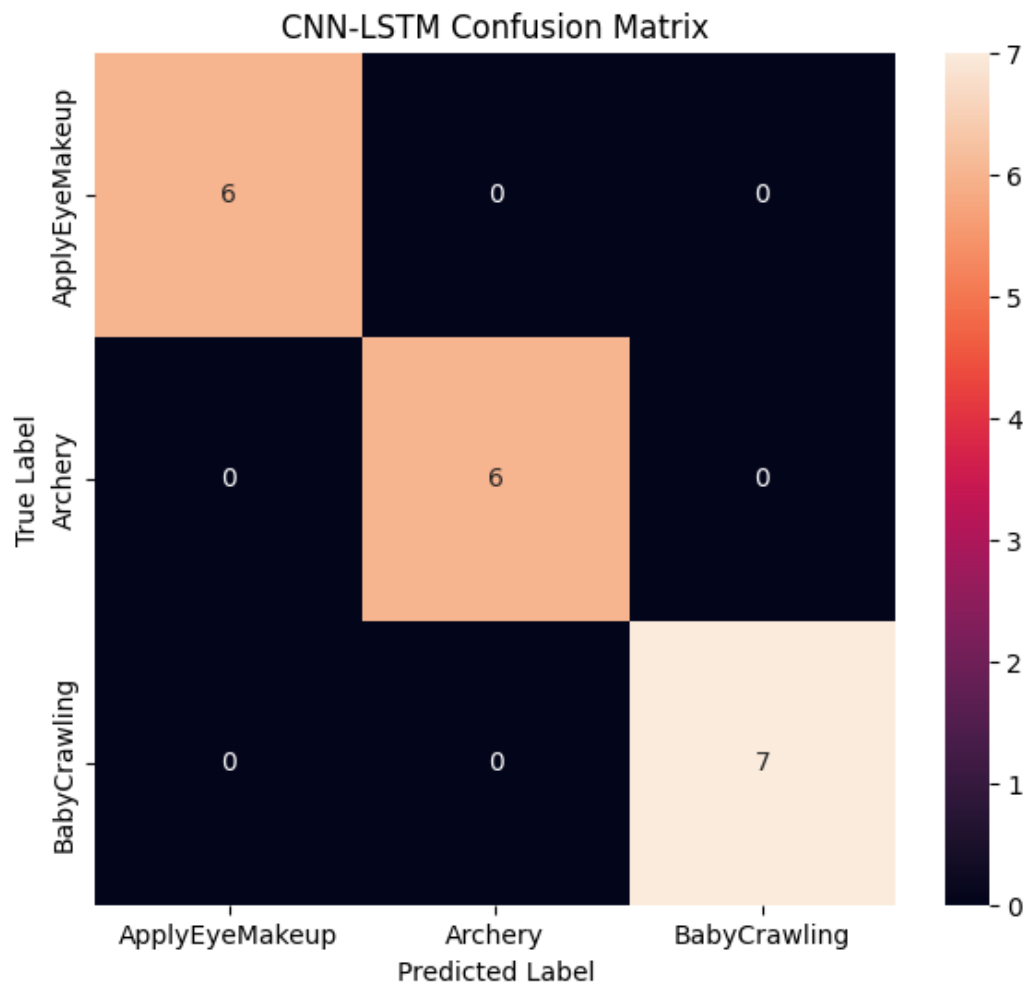


Figure 16: CNN-LSTM confusion matrix.

Test accuracy, macro precision, recall and F1 were all 100%. The model had 168,643 trainable parameters and

required 5.69 seconds of recurrent-classifier training.

CNN-GRU

The CNN feature sequence was kept identical and only the recurrent classifier was replaced with GRU(32).

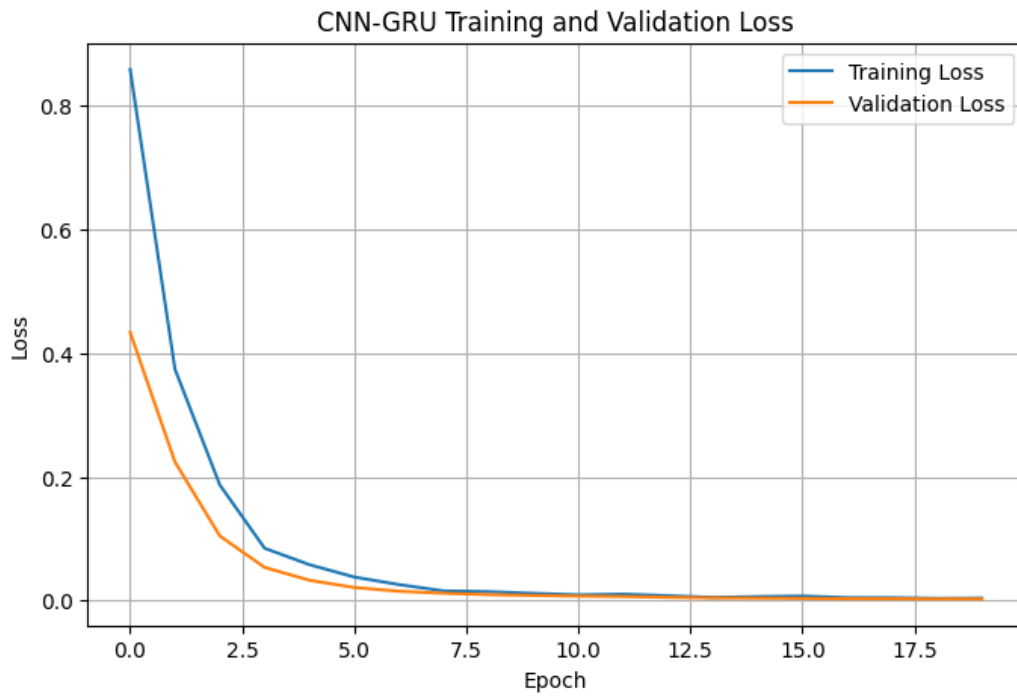


Figure 17: CNN-GRU training and validation loss.

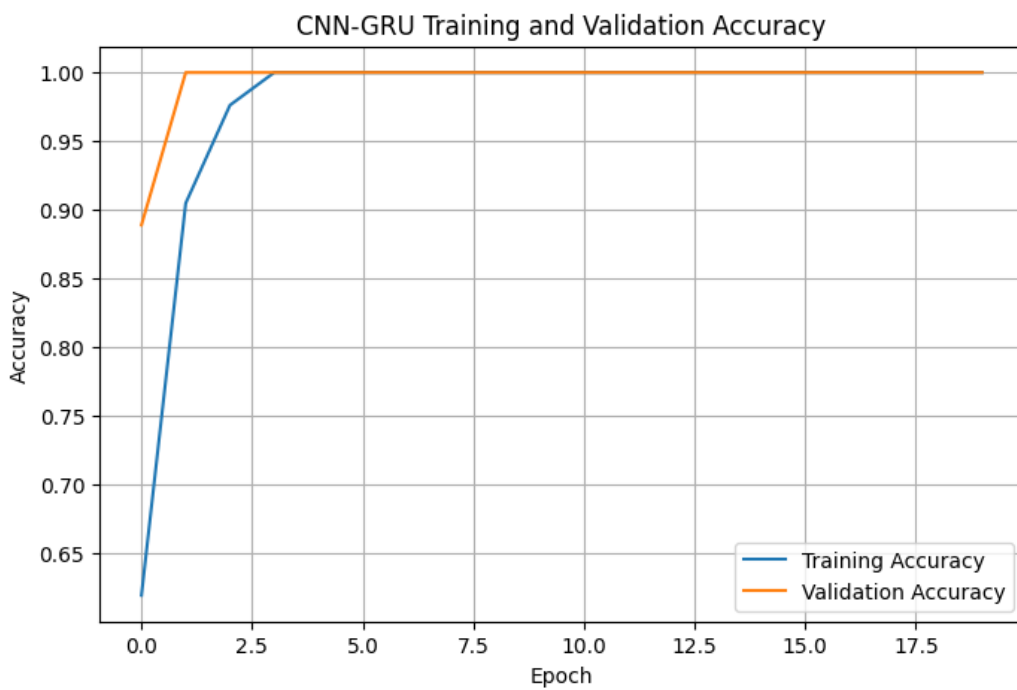


Figure 18: CNN-GRU training and validation accuracy.

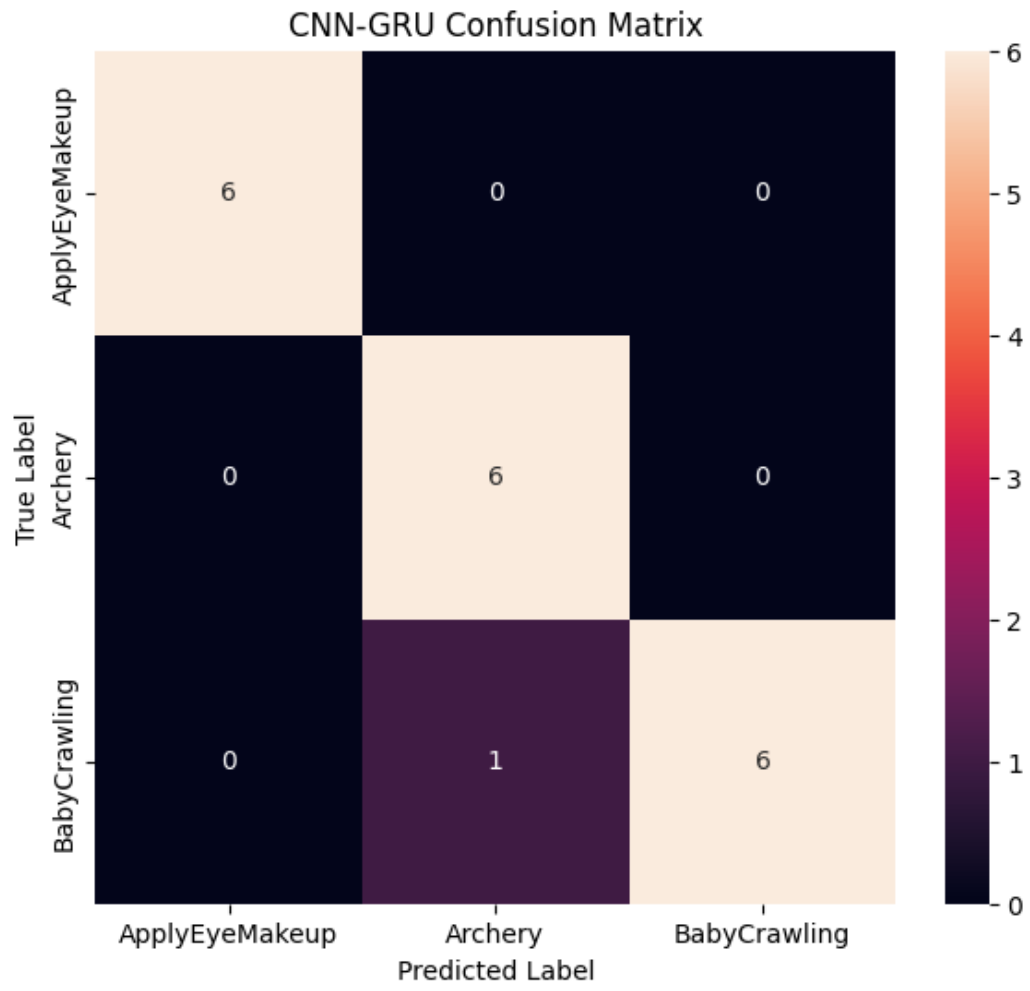


Figure 19: CNN-GRU confusion matrix.

Model	Accuracy	Precision	Recall	Macro F1	Parameters	Time (s)
CNN-LSTM	100.00%	100.00%	100.00%	100.00%	168,643	5.69
CNN-GRU	94.74%	95.24%	95.24%	94.87%	126,723	7.77

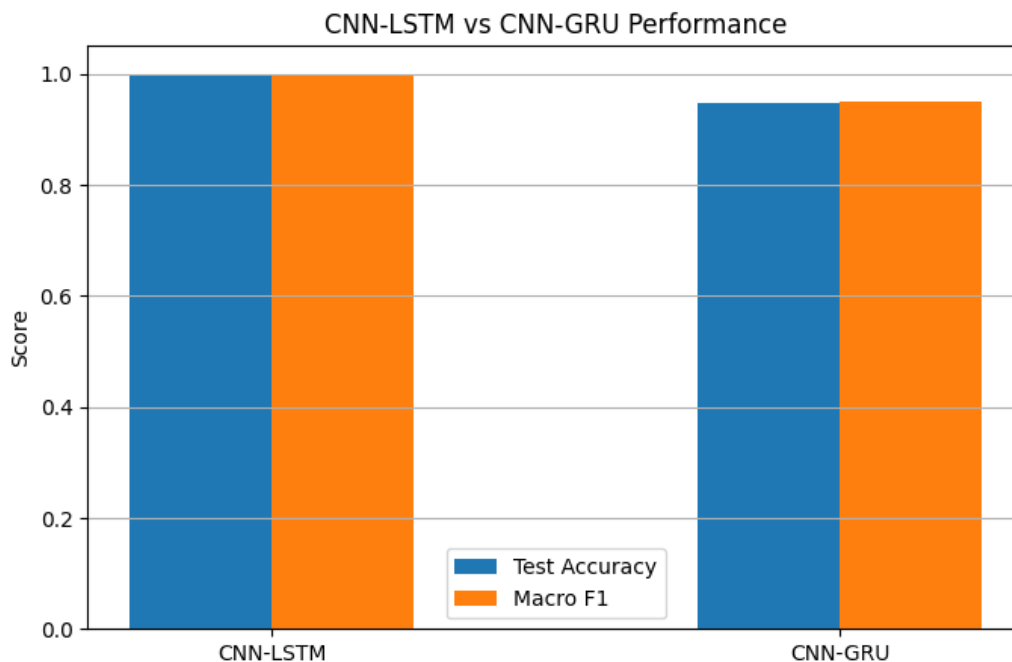


Figure 20: CNN-LSTM and CNN-GRU performance comparison.

The CNN-GRU confusion matrix shows one BabyCrawling sample predicted as Archery in the 19-video test set. The CNN-LSTM test set predictions were all correct in this execution. Because the video test set is small, these results should be interpreted as results for the selected laboratory subset rather than a broad estimate of UCF101 performance.

12. Sequence-to-Sequence Learning

A synthetic reversal task was generated using 2,000 sequences of length four with integer tokens from 1 to 9. For example,

$$[7, 4, 8, 5] \rightarrow [5, 8, 4, 7].$$

A start token of zero was used for decoder input. The encoder consisted of an embedding layer followed by an LSTM with latent dimension 64. The decoder used a second embedding layer, LSTM and a softmax output layer.

Input Sequence $\rightarrow$ Encoder LSTM $\rightarrow$ Context States $\rightarrow$ Decoder LSTM $\rightarrow$ Output Sequence.

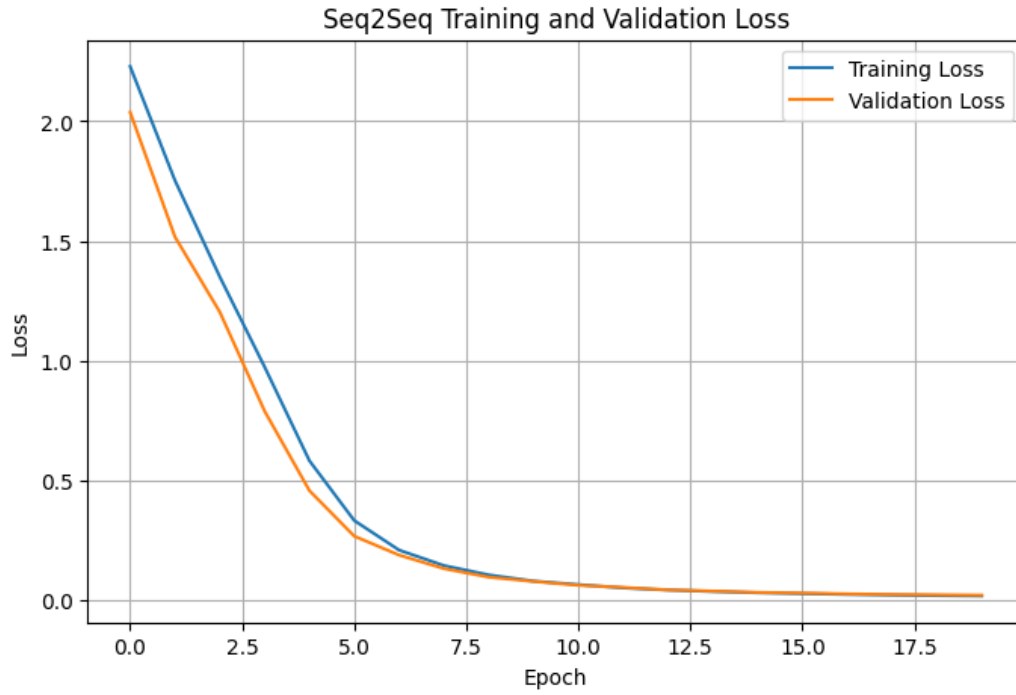


Figure 21: Sequence-to-sequence training and validation loss.

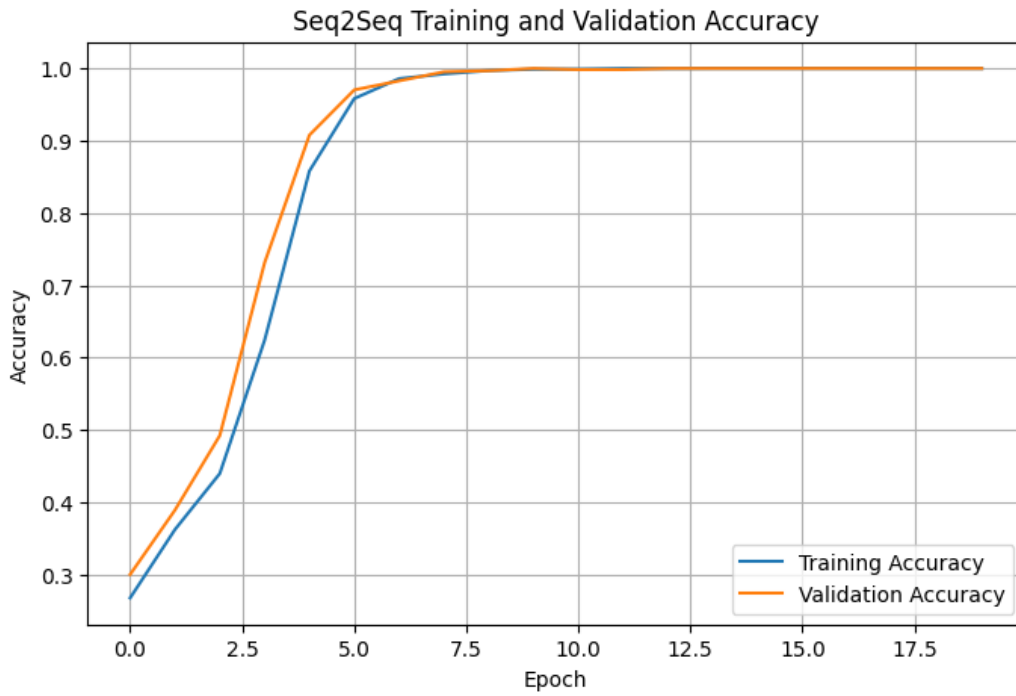


Figure 22: Sequence-to-sequence training and validation accuracy.

Metric	Result
Token Accuracy	99.9375%
Sequence Accuracy	99.7500%
Training Time	18.34 s
Trainable Parameters	50,954

Five test examples were displayed in the notebook. Token accuracy measures individual token correctness, whereas sequence accuracy requires every token in a complete sequence to be correct. Therefore a sequence can have high token accuracy but still be counted as incorrect at the sequence level if even one token is wrong.

13. Consolidated Results

Model	Accuracy	Precision	Recall	F1	Parameters	Time (s)
Vanilla RNN	80.00%	78.94%	79.11%	78.99%	1,974	78.61
LSTM	94.89%	95.23%	95.22%	95.22%	6,006	78.86
GRU	95.34%	95.66%	95.68%	95.66%	4,758	73.95
CNN-LSTM	100.00%	100.00%	100.00%	100.00%	168,643	5.69
CNN-GRU	94.74%	95.24%	95.24%	94.87%	126,723	7.77

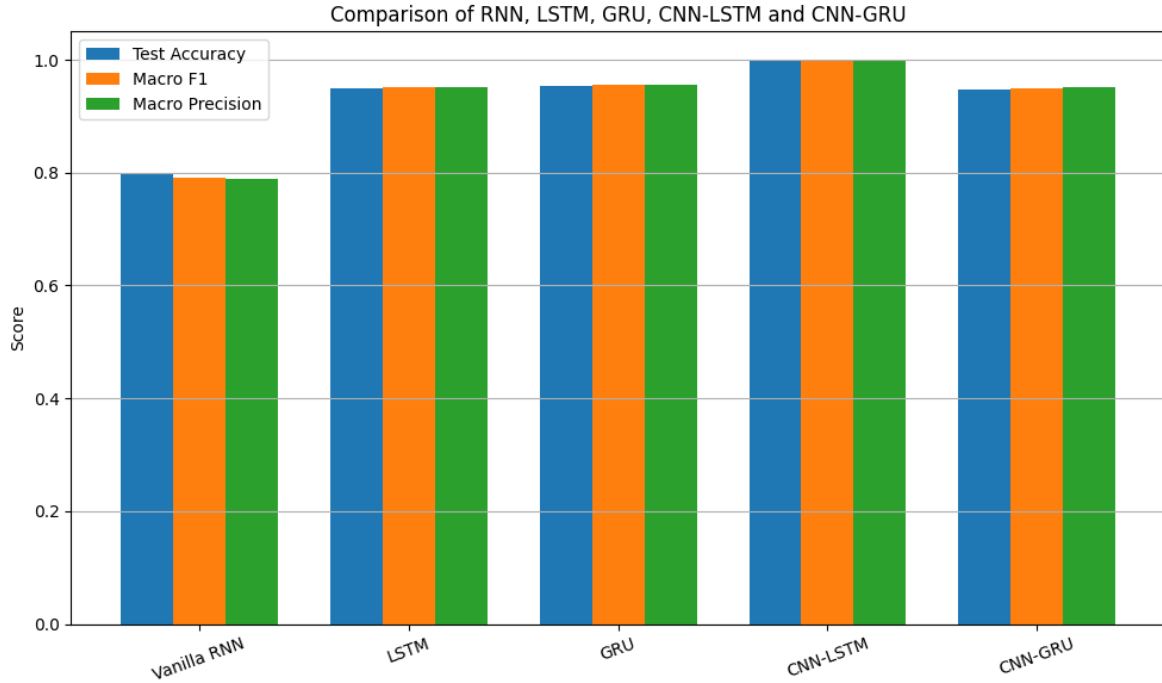


Figure 23: Consolidated model comparison.

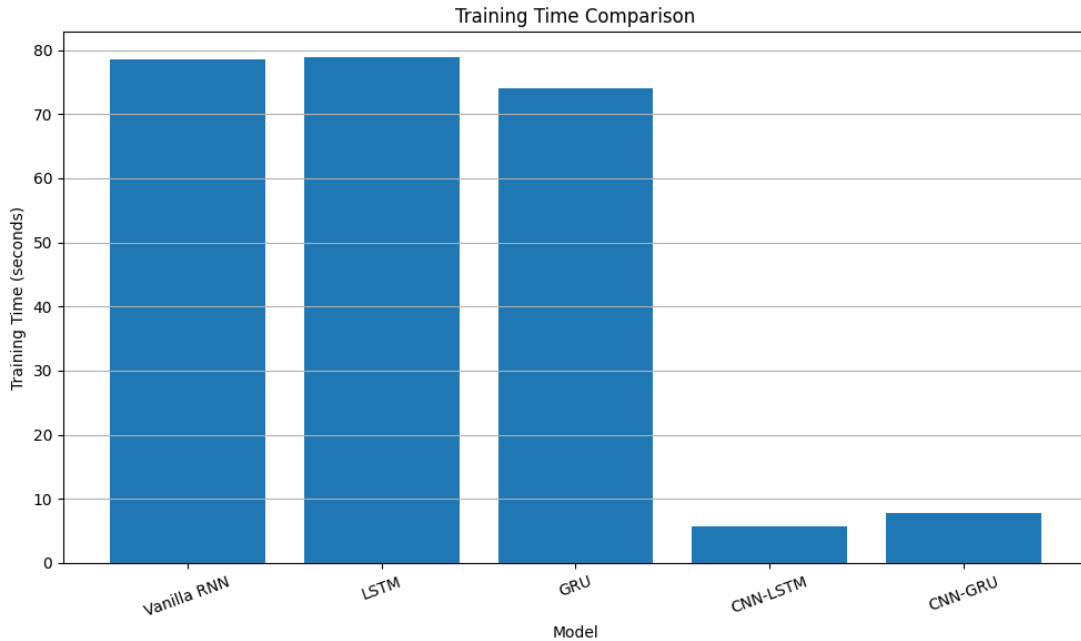


Figure 24: Training time comparison.

Overall inference: On the HAR dataset, the gated recurrent models substantially outperformed the Vanilla RNN in this execution. GRU used fewer parameters than LSTM and achieved a slightly higher Macro F1 while also taking less training time. In the video experiment, the CNN converted individual frames into compact spatial feature vectors, after which the recurrent layer modeled the temporal sequence of those vectors.

14. Required Inferences

1. **Temporal pattern:** Sensor channels show changing values across the 128 time steps, with activity-dependent temporal patterns.
2. **RNN convergence:** The RNN learned progressively but showed lower final test performance than the gated models.
3. **LSTM convergence:** LSTM reached high validation accuracy early and maintained strong performance.
4. **GRU convergence:** GRU maintained consistently high validation accuracy and achieved the highest HAR Macro F1 among the three recurrent models in the final comparison.
5. **Overfitting/generalization:** Training and validation curves should be considered together. In the recurrent experiments, validation performance remained relatively close to training performance for the better-performing gated models.
6. **Confusion-matrix errors:** The RNN showed more errors in dynamic activities, while LSTM and GRU reduced these errors substantially. The exact class-wise errors are visible in the included matrices.
7. **Sequence length:** The effect depends on the architecture; RNN performance decreased with longer sequences in this experiment, while LSTM and GRU remained stable.
8. **CNN role:** The CNN extracts spatial visual features independently from individual frames.
9. **Recurrent role:** LSTM/GRU learns temporal relationships between the ordered frame-level feature vectors.
10. **Token vs sequence accuracy:** Sequence accuracy is stricter because a complete sequence is correct only when all its tokens are correct.

15. Discussion Questions and Answers

1. **What is an RNN?** An RNN is a neural network designed for sequential data that maintains a hidden state carrying information from previous time steps.
2. **What does the hidden state represent?** It represents a learned summary of the current input together with relevant information from previous time steps.
3. **Why is sequence order important?** The temporal ordering contains information about how the underlying process evolves over time.
4. **Feed-forward network vs RNN:** A feed-forward network processes inputs without recurrent state, whereas an RNN reuses recurrent weights and maintains a hidden state across time steps.
5. **What is BPTT?** Backpropagation Through Time unrolls the recurrent network over time and propagates the loss gradient backward through those recurrent steps.
6. **Why do gradients vanish?** Repeated multiplication by derivatives with magnitude below one can make the gradient shrink toward zero.
7. **Why do gradients explode?** Repeated multiplication by factors with sufficiently large magnitude can make gradient values grow very large.
8. **What are long-term dependencies?** They are relationships where information from an earlier time step is useful for interpreting or predicting a much later time step.
9. **Role of LSTM cell state:** It provides a memory pathway that allows information to be selectively retained or discarded.
10. **LSTM gates:** The forget, input and output gates control removal, addition and exposure of information respectively.
11. **GRU gates:** The reset gate controls how previous state contributes to the candidate state, while the update gate controls the combination of previous and candidate states.
12. **LSTM vs GRU:** LSTM has a cell state and three principal gates; GRU uses a hidden state with two principal gates and a simpler structure.
13. **Why is GRU simpler?** It combines memory-control functions into fewer gates and does not maintain a separate cell state.
14. **Why use identical settings?** Keeping input, optimizer, batch size and evaluation protocol fixed makes the model comparison controlled.
15. **What does a confusion matrix reveal?** It shows class-wise correct predictions and which classes are being confused, information that overall accuracy alone does not provide.
16. **Why macro F1?** Macro averaging gives each class equal weight and is useful for multi-class class-wise comparison.
17. **Significance of sequence length:** It determines how much temporal context is available to the recurrent model.
18. **Why use CNN for video?** A CNN efficiently converts each image frame into a learned spatial feature representation.
19. **What does the CNN extract?** It extracts spatial and visual features from individual frames.
20. **What does the recurrent network learn?** It learns temporal relationships among the ordered frame features.
21. **Why arrange CNN features as a sequence?** A video consists of ordered frames, so the frame-level feature vectors form a temporal sequence suitable for LSTM or GRU processing.

22. **Encoder and decoder:** The encoder converts the input sequence into a learned state representation, and the decoder generates the output sequence from that representation.
23. **What is teacher forcing?** During decoder training, the previous ground-truth output can be supplied as the next decoder input instead of the previous predicted token.
24. **Token vs sequence accuracy:** Token accuracy evaluates individual positions, whereas sequence accuracy requires the entire predicted sequence to be correct.
25. **Why keep the test set untouched?** To prevent model selection and tuning decisions from using information from the final evaluation set.

16. Conclusion

The experiment demonstrated an end-to-end sequence-learning workflow using raw smartphone sensor signals. The Vanilla RNN achieved 80.00% test accuracy, while LSTM and GRU achieved 94.89% and 95.34%, respectively. The results illustrate the practical benefit of gated recurrent mechanisms for this sequence-classification task. The sequence-length experiment showed that the effect of temporal context depends on the recurrent architecture.

The video extension demonstrated the spatial-temporal pipeline: MobileNetV2 extracted a 1280-dimensional feature vector from each of ten sampled frames, and the recurrent classifier modeled the resulting sequence. The synthetic sequence-to-sequence experiment achieved 99.9375% token accuracy and 99.7500% sequence accuracy, demonstrating the encoder-decoder formulation.

17. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” *EMNLP*, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” *ESANN*, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>

- NAME: T Pranav Varshin(24110413)
- Reg No: 24011101116
- Section: AIDS-B